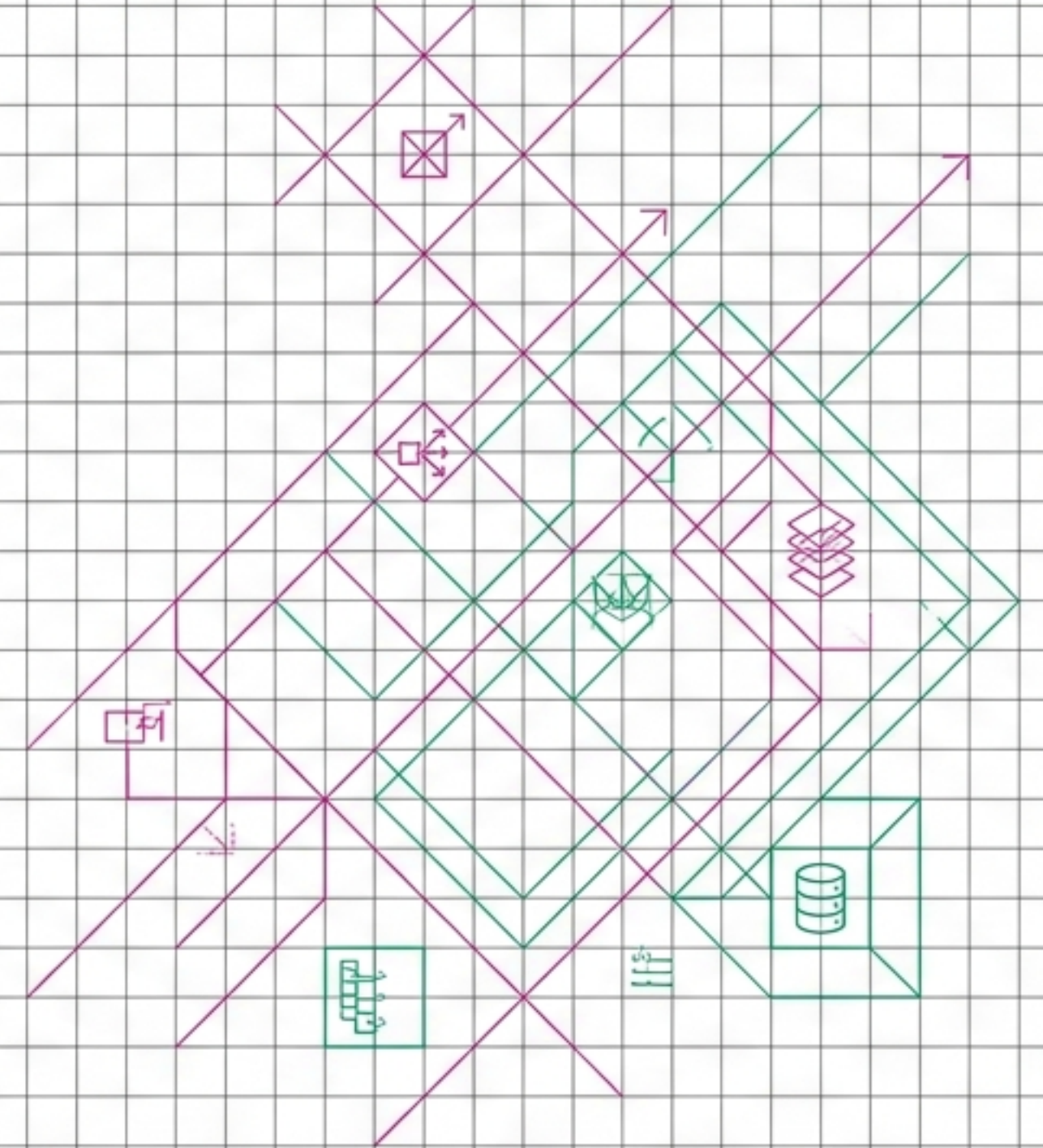


CampusLab — Arquitectura Cloud Native Segura

Evaluación EP2 · Caso 2 · DuocUC DSY1107

Katherine Ramirez, Marcelo Prado y Williams Rivas



Contexto y Objetivos del Sistema

CampusLab: Plataforma centralizada para la reserva de laboratorios y equipos académicos. Objetivo principal: Desplegar un sistema funcional y seguro en la nube, protegido por la identidad corporativa de Microsoft.

Frontend
React + Vite

Backend
Spring Boot (BFF +
Microservicios)

Infraestructura
EC2 + Docker Compose

Alcance Técnico del Caso 2

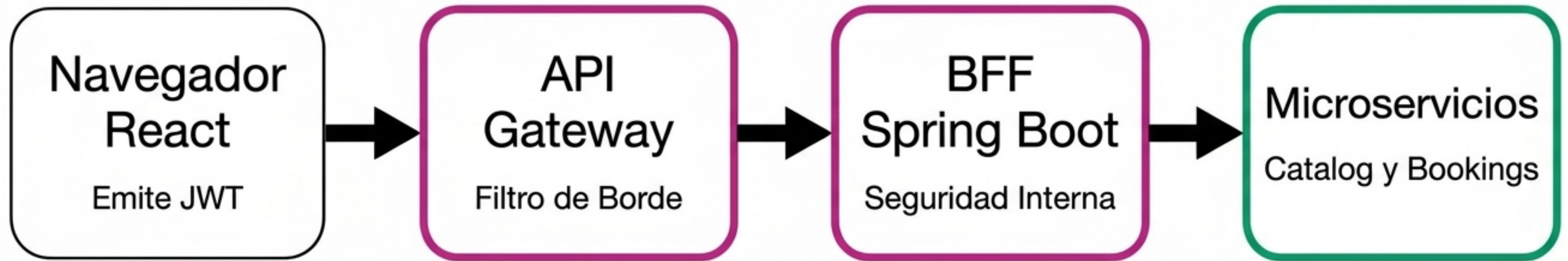
Incluido en la Arquitectura

- Identidad federada (Entra ID + Cognito)
- API Gateway (HTTP API)
- BFF con Spring Security
- Microservicios (Catalog & Bookings)

Fuera de Alcance (Excluido)

- Mensajería Kafka / Streaming
- KPIs y Analítica
- Auditoría por streaming
- AWS Lambda (Integración HTTP directa al BFF)

La Regla de Oro: El Flujo Seguro de la Petición



Enrutamiento estricto: Integración HTTP directa sin uso de AWS Lambda.

Identidad Federada: Entra ID + Amazon Cognito



Proveedor OIDC configurado hacia Entra ID.
User Pool: us-east-1_dBOQeniQq.
Define grupos de acceso: Admin / Operador.

App Registration en tenant corporativo
d84b2661 (WRIVAST).
Delega la validación de credenciales.

El login se delega completamente en Microsoft; Cognito actúa como intermediario para emitir el token estandarizado.

Flujo OIDC: Authorization Code con PKCE

```
1 {  
2   "token_use": "access",  
3   "client_id": "6r3af8ucc96okla0ujph9pru4a",  
4   "cognito:groups": [ "Admin" ],  
5   "username": "demo.admin"  
6 }
```



App Client Público: Sin secreto alojado en el navegador.

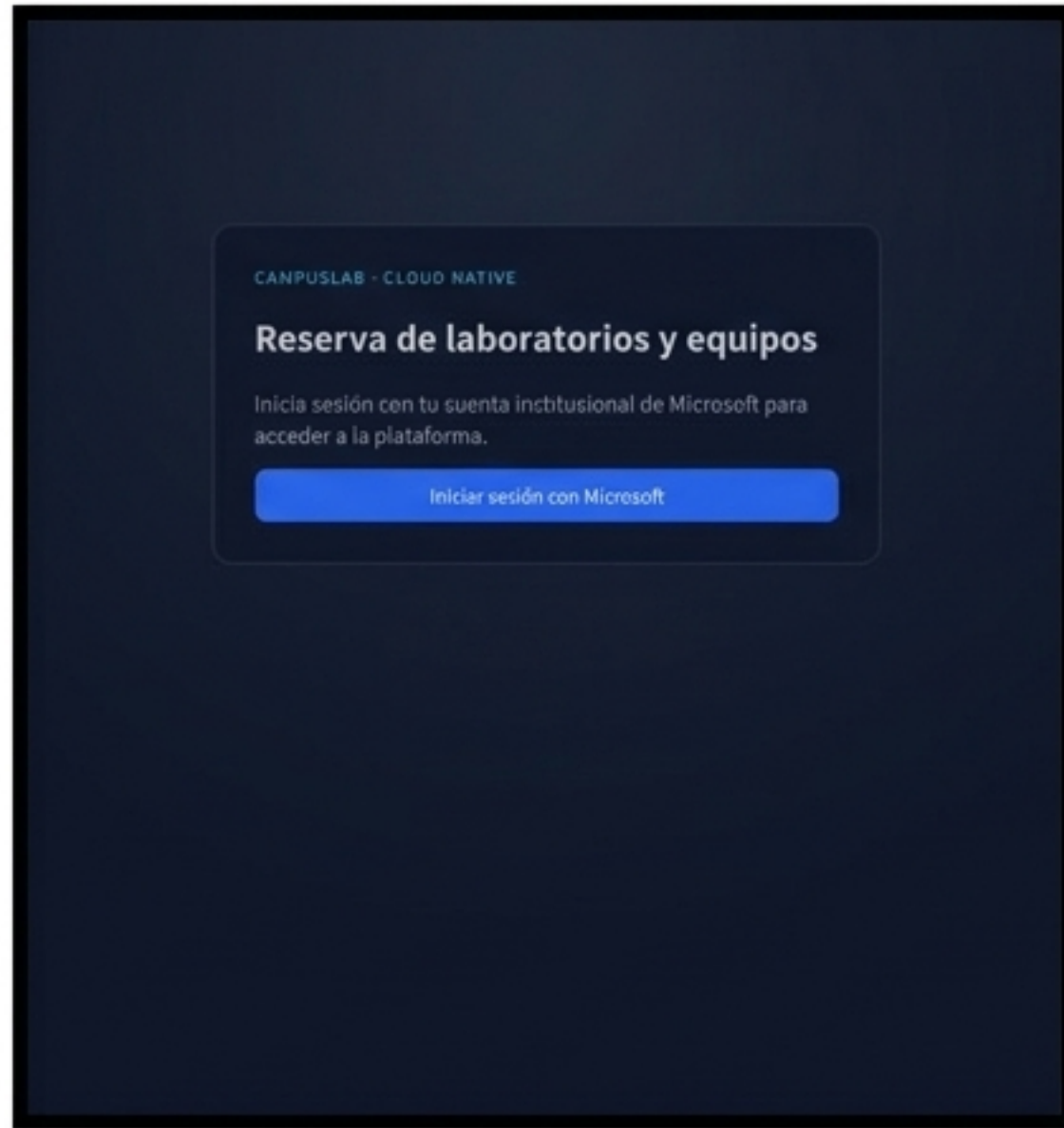


S256 Hash: El cliente React genera dinámicamente el code_verifier y code_challenge.

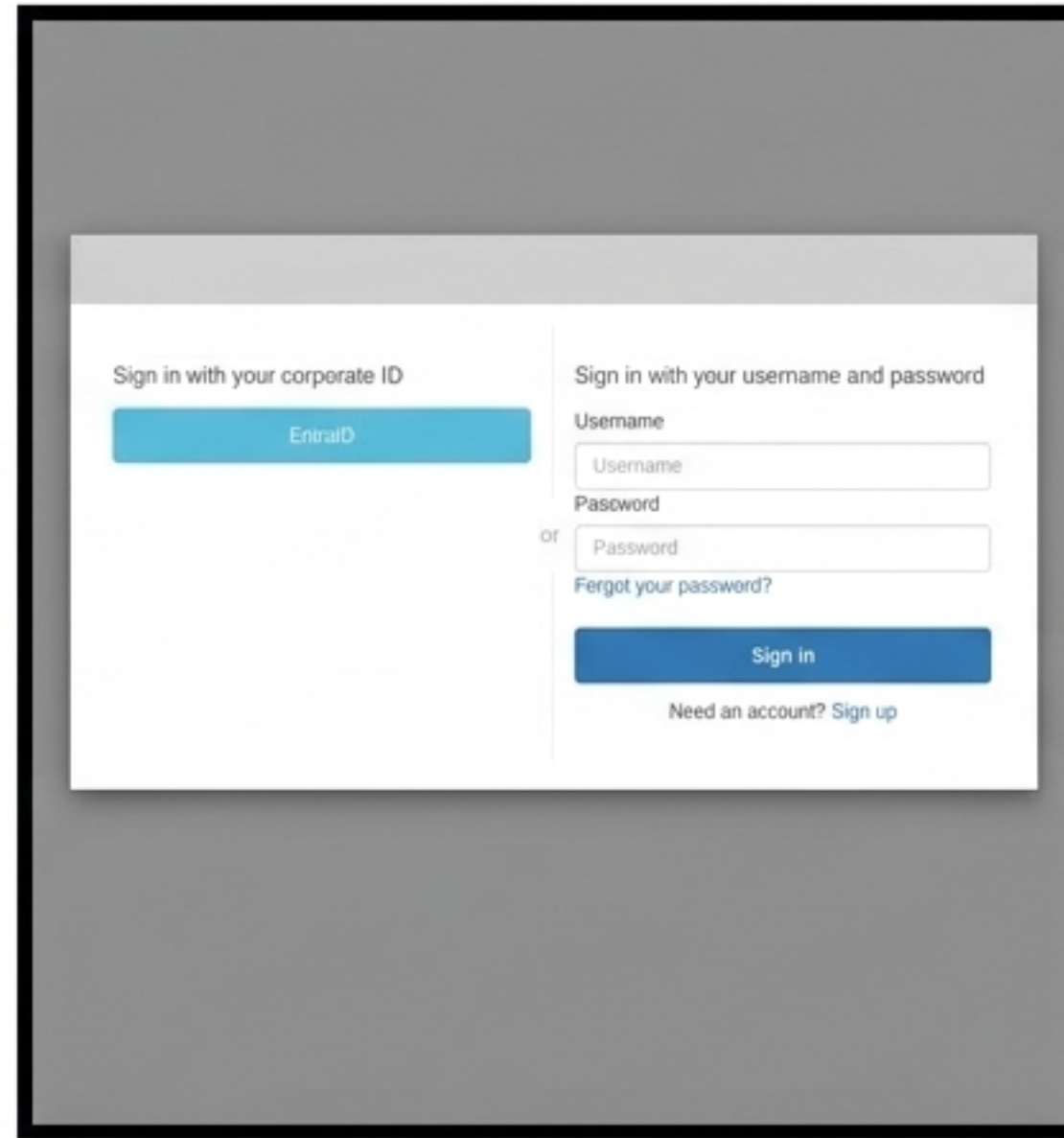


Headers: El frontend adjunta Authorization: Bearer en cada llamada subsiguiente.

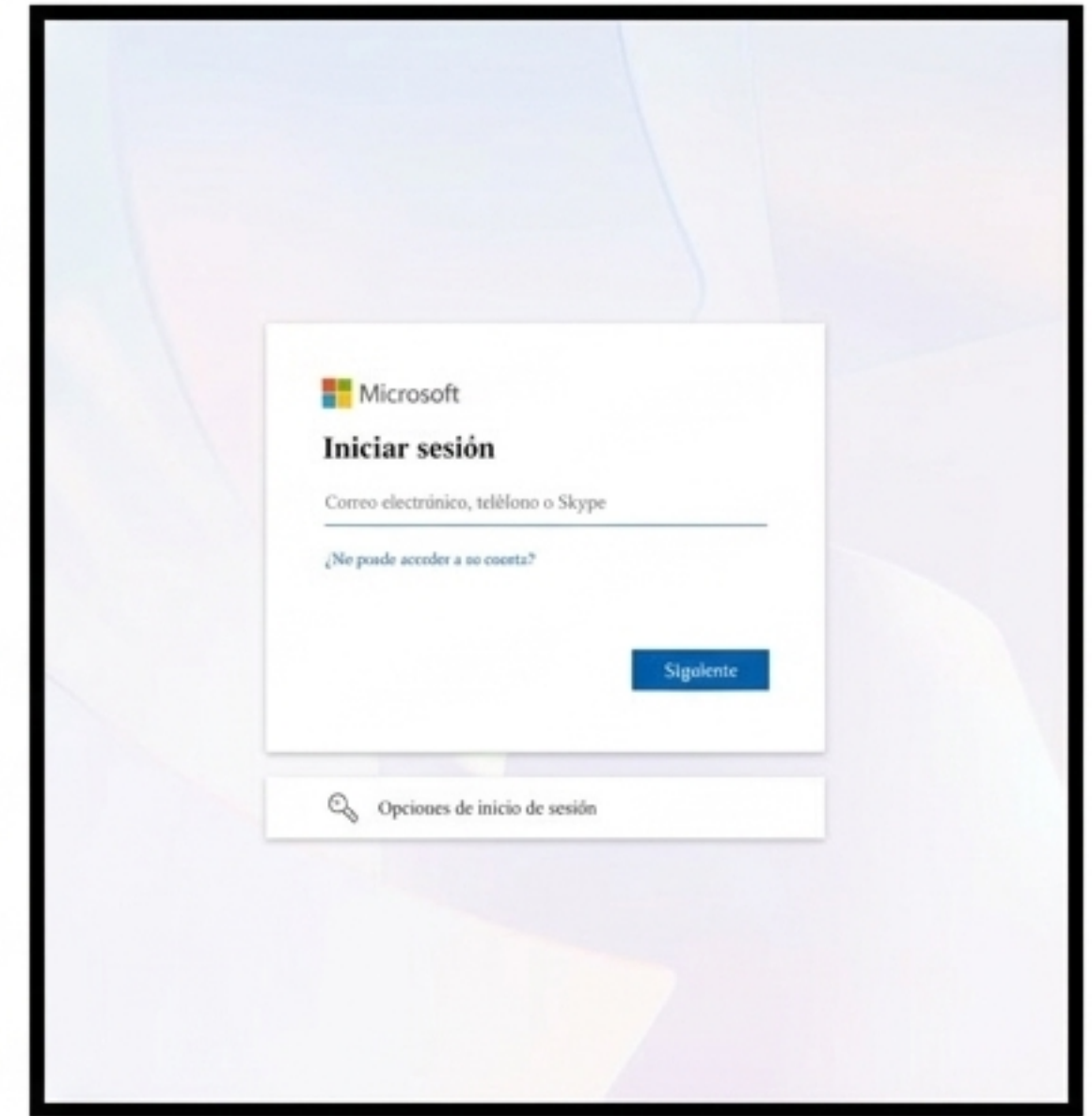
Evidencia Real: Experiencia de Login Federado



1. Inicio en React



2. Redirección
Cognito OIDC



3. Autenticación en
Tenant WRIVAST

Primer Control: Validación JWT en el API Gateway

HTTP 401

`GET /api/catalog/resources`

Petición sin Token.

Resultado: Rechazado en el borde.
El tráfico malicioso nunca toca la
infraestructura EC2.

HTTP 200

`GET /api/catalog/resources`

Petición con Token Admin.

Resultado: Autorizado. El JWT
Authorizer valida Issuer y Audience
(client_id).

Segundo Control: Autorización por Rol en el BFF



HTTP 403 Forbidden

Request:	POST /api/catalog/resources
Actor:	Usuario autenticado (token válido), pero SIN el rol Admin.
Mecanismo:	El BFF revalida la firma del token y extrae los claimsognito:groups. Al no encontrar el rol requerido, Spring Security rechaza la petición. Desacoplamiento total de seguridad.

Matriz CRUD Extremo a Extremo

Endpoint	Actor	Resultado
GET /api/catalog	Admin	200 OK
POST /api/catalog	Admin	201 Created
POST /api/catalog	No Admin	403 Forbidden
PUT /api/bookings/{id}/status	Admin	200 OK
DELETE /api/bookings/{id}	Admin	204 No Content

Batería ejecutada contra el Invoke URL real afectando contenedores en EC2.

Integridad de Datos: Validación Estricta de Esquema

Input	Output
POST /api/bookings Payload con RUT mal formateado y sin nombre.	HTTP 400 Bad Request <pre>{ "errores": { "rut": "El rut debe tener formato 12345678-9 (sin puntos)", "nombre": "El nombre es obligatorio" } }</pre>

Validación a nivel de microservicio (Spring Boot), protegiendo la persistencia JPA.

Lógica de Negocio: Máquina de Estados de la Reserva



Regla estricta aplicada por el backend: El sistema rechaza cualquier transición a EN_USO si la reserva no se encuentra previamente APROBADA.

Despliegue Cloud: Instancia EC2 y Docker Compose

```
$ sudo docker compose ps
```

NAME	STATUS	PORTS
campuslab-catalog	Up (healthy)	8081
campuslab-bookings	Up (healthy)	8082
campuslab-bff	Up (healthy)	0.0.0.0:8080->8080

AWS EC2: Amazon Linux 2023
(IP: 3.237.97.7)

Security Groups: Puertos
estrictamente limitados a 22
(SSH), 8080 (BFF) y 8090.

Conclusiones de Arquitectura

- ✓ **Identidad Robusta:** Integración sin fricciones entre Microsoft Entra ID y Cognito (OIDC + PKCE).
- ✓ **Seguridad en Capas:** JWT validado en el borde (API Gateway) y autorización fina por rol en el BFF.
- ✓ **Despliegue Nativo:** Microservicios Microservicios Spring Boot ejecutándose en contenedores saludables sobre AWS EC2.

Código fuente y documentación completa:
github.com/WilliamsRT93/campuslab-caso2